Science Canvas Creator - Image Storage Optimization

Date: 15 February 2026 **Total Price:** $395 USD **Total Timeline:** 3-4 days

---

Milestone: Asset-by-Reference System

Price: $395 **Timeline:** 3-4 days

Important Consideration:

The process described in this milestone will optimize any images newly added to projects. Images previously embedded in existing projects will still contain complete image data. The milestone scope can be expanded to also update all existing projects.

Current State

Images added via the Image tool or Assets tab are automatically uploaded to Supabase Storage and appear in the Assets library. However, when added to the canvas, the full image content (base64 data URL or SVG data) is embedded directly in the project save data instead of being referenced. This causes save file bloat - a single embedded image consumes ~650 KB of the 4MB limit.

Library icons use an optimized icon-by-reference system that stores only `iconId` + transforms in the project data and fetches the icons from the library on project load. This same reference-based approach needs to be applied to user-uploaded images.

Deliverables

Reference-Based Image Storage

- Images store references (asset ID) instead of embedded content in project save data
- Projects load images from storage and reconstruct canvas content using saved references (same pattern as library icons)

- Image tool displays loading placeholder while uploading, then replaces with uploaded image on canvas
- Images deleted from canvas remain in Assets library for reuse
- Images deleted from Assets library show confirmation warning that they will be removed from all projects using them

Complete Image Feature Compatibility

- All existing image features work with optimized storage: cropping, filters, color maps, masking, opacity, blending modes
- Copy/paste preserves all image properties and references
- Undo/redo works correctly with stored images
- Image quality and aspect ratio maintained
- Cropped, filtered, and masked images save and reload with all edits preserved
- Original image data is stored for non-destructive editing

Updated Thumbnail Generation

- Thumbnail generation fetches and renders user assets from storage (same pattern as library icons)
- Project thumbnails accurately reflect canvas content with referenced images

Storage Quota Management

- 500 MB storage limit per user
- Storage usage indicator in Assets tab showing used/total space
- Upload blocked when quota exceeded with clear error message
- Real-time storage usage tracking

Vercel SPA Configuration Fix

- Update configuration to enable both Vite and Vercel to handle client-side routing
- Page refreshes on any route return the app instead of 404 errors
- Direct navigation to client-side routes works correctly

Tests

Unit Tests

Save/Load Serialization:

- Assets have `src` property stripped on save, serialized data <1 KB
- Save data includes `assetId` and all image properties (crops, filters, masking, transforms)
- Load fetches asset content from storage and reconstructs with saved properties

Storage Quota Enforcement:

- Calculate total storage usage from multiple assets correctly
- Quota check returns true when under limit (e.g., 400 MB used / 500 MB limit)
- Quota check returns false when at/over limit (e.g., 500 MB used / 500 MB limit)
- Storage usage updates correctly after asset deletion

End-to-end Tests

Image Tool Upload:

- Click Image tool → Select image → Loading placeholder appears on canvas → Image uploads to storage → Placeholder replaced with uploaded image
- Uploaded image appears in Assets tab
- Works for JPG, PNG, and SVG formats

Assets Tab Integration:

- Click existing asset in Assets tab → Asset added to canvas with assetId reference
- Both Image tool and Assets tab produce identical asset-referenced objects

Asset-by-Reference Save/Load:

- Save project with 10 user assets → Canvas data <1 MB
- Load project → All assets render correctly
- Format-specific round-trip: JPG asset → Save → Reload → Renders correctly; PNG with transparency → Save → Reload → Transparency preserved; SVG → Save → Reload → Renders as vector
- Image properties preserved: crops, filters, color maps, masking
- Copy/paste and undo/redo work correctly

Missing Asset Handling:

- Load project with missing asset (deleted from storage) → Placeholder shown with clear warning message

Asset Deletion:

- Delete image from canvas → Image remains in Assets tab

- Delete image from Assets tab → Confirmation shows "This will remove the image from all projects using it"
- Confirm deletion → Image removed from storage and projects

Storage Quota UX:

- Assets tab displays storage indicator showing "X MB / 500 MB used"
- Upload new image → Storage indicator increases by image file size
- Delete image from Assets tab → Storage indicator decreases
- When quota exceeded: Upload blocked with "Storage quota exceeded" error message and guidance to free up space

Manual Tests

Thumbnail Generation:

- Generate thumbnail for project with 5 asset references → All assets render correctly in thumbnail
- Thumbnail generation fetches assets from storage (not using embedded data)

Storage Quota Exceeded:

- Upload files to reach quota limit → Verify blocking works correctly
- Error message provides clear, helpful guidance to free up space
- Quota enforcement feels responsive (not delayed/buggy)

Asset Deletion UX:

- Confirmation message clearly states "This will remove the image from all projects using it"
- Warning is prominent and prevents accidental deletion

Vercel SPA Routing:

- Navigate to `/canvas/project-123` → Refresh page → App loads correctly (no 404)
- Direct navigation to any client-side route works in production deployment
- Browser back/forward navigation works correctly

Notes

Development Database

An independently owned and managed development database will be provided for development and testing. Migrations, edge functions, and configuration will be deployed to the client's production database when each milestone is submitted. A seed script will be used to populate the development database with placeholder data for testing.

GitHub Repository

The project repository will be cloned into an independently owned and managed private GitHub repository for development isolation. Changes will be pushed to the production repository when each milestone is submitted for review.

Preview deployment

A preview deployment connected to the development database will be provided with the first milestone submission. This preview will be updated throughout the development process via continuous deployment of the development GitHub repository.

Production deployment and database

Production deployment and database hosting can be directly managed by the developer via Vercel or Supabase (an invitation as a team member is required). Alternatively, detailed documentation can be provided, along with ongoing guidance and support via Upwork chat messaging.

API Credentials and Integrations

Independently owned and managed API keys will be provided for development and testing (example: Resend, OpenRouter, Stripe, Twilio, SendGrid, OpenAI). Production accounts and API keys for external integrations will be provided by the client. Detailed documentation and ongoing guidance and support via Upwork messaging can be provided to assist in implementing and managing the accounts and configurations of any external integrations. Direct account management for these external services is not provided by the developer.

Delivery Report

A delivery report will be provided when each milestone is submitted for approval. This report will include comprehensive test results for all tests related to the milestone scope. All milestone tests will be executed against the development database environment. The

delivery report will describe the work completed and provide any additional context related to the development process and the work delivered.

Post-Delivery Support

Ongoing messaging support, bug fixes, and issue resolution is provided for all unaltered code and related database infrastructure within the scope of completed milestones. This support applies exclusively to the codebase and database in their delivered state. Any modifications made after delivery, including changes to styling, configuration, or unrelated features, void this support as they cannot be validated against the delivered testing suite.

Milestone Scope

Each milestone is complete when all items in the **Deliverables** sections are implemented and all tests in the **Tests** section pass. If functionality wasn't explicitly included in the deliverables or validated by a Test, then it is outside of the milestone scope.

Contract Completion

Completion of all deliverables described in all milestones constitutes completion of the contract. No additional tasks beyond the scope directly specified within each milestone are required for the contract to be considered successfully completed. Additional milestones can be added if further work is required, provided that both parties agree to the expansion of the contract scope.

Testing Philosophy

Testing is designed to verify that specified test cases pass and does not guarantee that the application is free of all defects or will meet all user expectations. Test coverage will be as comprehensive as reasonably achievable within the defined scope. Edge cases and scenarios not explicitly described in the deliverables may not be covered unless they emerge during testing.